Introducción a C# .NET

Tecnicatura Superior en Programación
Materia: Programación III
Docente: Alejandro Tassoni 2 Nivel
UTN – FRRA

Temas

Qué es C#?

Estructura de un programa C#

Tipos de Datos

Variables

Operadores

Control de Flujo

¿ Qué es C#?

Lenguaje de propósito general, creado por Microsoft

Lenguaje evolucionado de C++ y Java

Ideal para .NET

Características de C #

- Lenguaje de alto nivel y sencillo
- Orientado a objetos
- Lenguaje moderno
- Sintaxis casi igual a la de Java

Estructura de una clase en C#

code

Librerías usadas

```
//inclusión de libreriadeClases/namespace
using System;
using System.Collecctions.Generic;
using System.Text
```

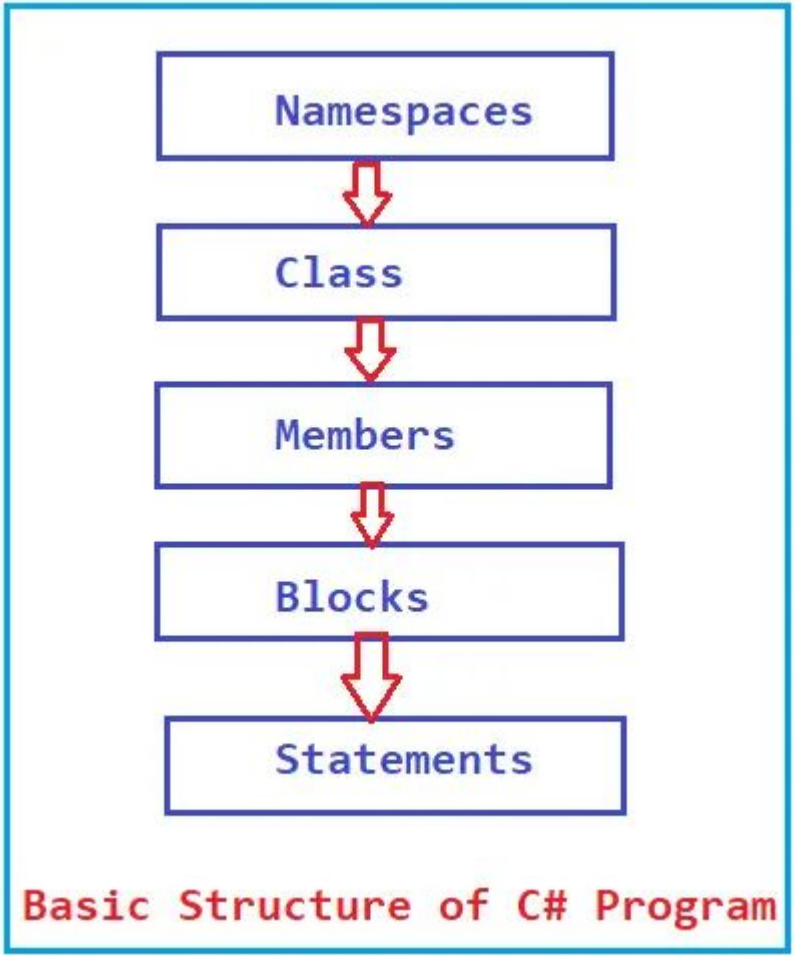
```
// declaración del namespace
namespace Miprograma
{
 // Declaración de Clase
Public Class Program{
      // comentario Simple
      /*
      Comentario Bloque
      */
      // Declaración de miembros
      propiedades

      // Funciones / Métodos /Bloques
      public static void Main(string[] arg)
      {
         //statements / declaraciones
         Console.WriteLine("Hola Mundo");
      }

       …
}
```

Método de Inicio

El proceso anterior se muestra en el siguiente diagrama.

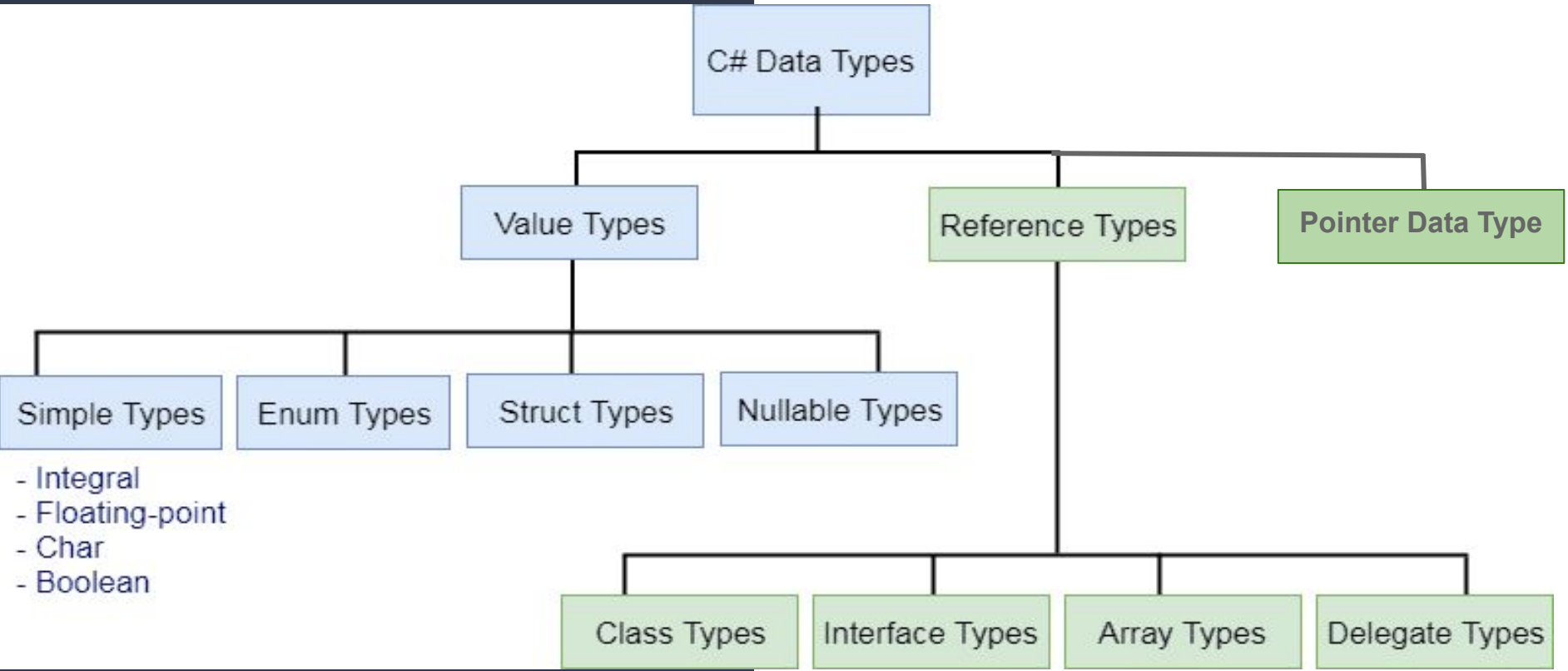


Tipos de Datos

Los tipos de datos son algo que da información sobre

- **Tamaño** de la ubicación de la memoria.
- El **rango de datos** que se pueden almacenar dentro de esa ubicación de memoria.
- Posibles **operaciones legales** que se pueden realizar en esa ubicación de memoria.
- ¿Qué **tipos de resultados** surgen de una expresión cuando estos tipos se utilizan dentro de esa expresión?

Tipos en C#


C# Data Types
Value Types
Reference Types
Pointer Data Type
Simple Types
Enum Types
Struct Types
Nullable Types
- Integral
- Floating-point
- Char
- Boolean
Class Types
Interface Types
Array Types
Delegate Types

Tipo de datos de valor en C#

code

El tipo de datos que almacena el valor directamente en la memoria se denomina tipo de datos de valor en C#.

Tipos de datos predefinidos : el ejemplo incluye entero, booleano, booleano, largo, doble, flotante, etc.

Esto tipos de datos integrados podemos consultarlo en

Tipos integrados (referencia de C#)

Tipos de datos definidos por el usuario : el ejemplo incluye struct, enumeraciones, etc.

Tipos de estructura (Referencia de C#)
Tipos de enumeración (referencia de C#)

El espacio de nombres System es el espacio de nombres de la raíz de los tipos fundamentales de .NET contiene clases que representan los tipos de datos base.

Tipo de datos de referencia en C#

El tipo de datos que se utiliza para almacenar la referencia de una variable.Los tipos de datos de referencia se clasifican en 2 y son los siguientes.

Tipos predefinidos: los ejemplos incluyen objetos, cadenas y dinámicas.

Tipos definidos por el usuario: los ejemplos incluyen clases e interfaces.

Tipo de datos puntero en C#

El puntero en C# es una variable que tiene una dirección de memoria, también se le conoce como localizador o indicador.

Para obtener los detalles del puntero tenemos dos símbolos ampersand (&) y asterisco (*).

Ampersand (&): Operador de dirección. Se utiliza para determinar la dirección de una variable.

asterisco (*): Operador Indirecto. Se utiliza para acceder al valor de una dirección.

Ejemplo de Puntero

```
3   namespace TipodeDatosPuntero
4   {
5       class Program
6       {
7           static void Main(string[] args)
8           {
9               unsafe // codigo inseguro
10              {
11                  // declaracion de la varioble
12                  int number = 10;
13
14                  // almacenar la dirección de la variable number en
15                  // la variable puntero ptr
16                  int* ptr = &number;
17                  Console.WriteLine($"Valor / Value :{number}");
18                  Console.WriteLine($"Direccion/Address :{(int)ptr}");
19                  Console.ReadKey();
20              }
21          }
22      }
23  }
```

Literales en C#

code

Un literal es un valor que se expresa como sí mismo en lugar de como el valor de una variable

- Los valores fijos se denominan literales en C#.
- Literal es un valor que utilizan las variables.

Ejemplo 1 **int x = 47;**

x es la **variable**

47 es **literal** = **al valor**

Ejemplo 2 **int j = 0X123F**

j es la **variable**

0X123F es **literal**

valor = 4671

Literales Enteros

Decimal (Base-10): dígitos del 0 al 9. No se requiere ningún prefijo para el tipo decimal de literal.

Hexa-Decimal (Base-16): dígitos del 0 al 9 y caracteres de a f en mayúsculas como en minúsculas. aquí C# no distingue entre mayúsculas y minúsculas. El número hexadecimal debe tener el prefijo 0X o 0x. y sufijo F

Binario (0 y 1): dígitos permitidos son solo unos y ceros. El número binario debe tener el prefijo 0b.

También se puede usar un sufijo con literales enteros como U o u para números sin signo, mientras que l o L se usan para números largos.

Literales de punto flotante

De forma predeterminada, cada literal de punto flotante es de tipo *double* por lo tanto, no podemos asignar valores directamente a variables *float* y *decimal*.

Si desea asignar valores a una variable *float*, debe agregar el **sufijo f o F** al final del literal

Si desea asignar valores a una variable *decimal*, deberá agregar el **sufijo m o M** al final del literal.

Si no agrega nada al literal de punto flotante, entonces el literal de punto flotante será *double* de forma predeterminada.

Si lo desea, también puede especificar explícitamente un literal de punto flotante como tipo *double* con el **sufijo d o D**

Literales de caracteres

Los literales de caracteres en C# están entre comillas simples, por ejemplo, 'a' , se almacenan en una varible de tipo char. Un carácter literal puede ser:

- un carácter simple, por ejemplo, ' a'
- una secuencia de escape, por ejemplo, '\t' ,
- un carácter universal, por ejemplo, '\u02B0'

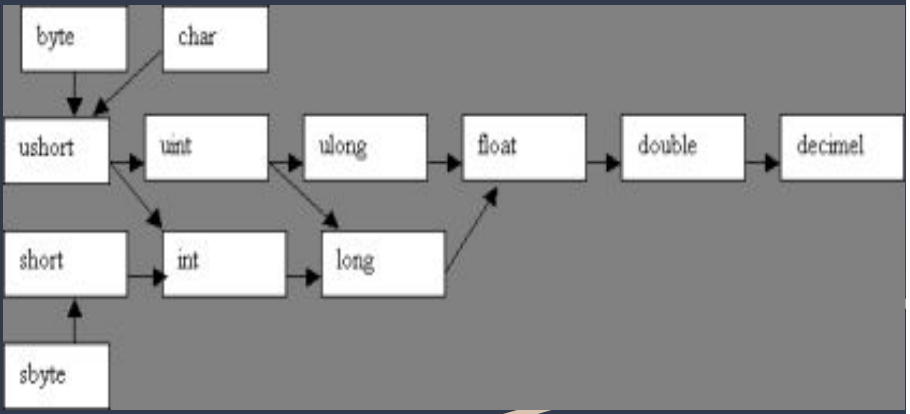
Entonces, podemos especificar literales de caracteres de 3 maneras. Son los siguientes:

- **char ch = 'A';**
- **char ch = '\n';** // linea nueva
- **char ch = '\u0041';** // Aquí /u0041 representa A

Tabla Caracteres de escape

Escape	Sequence Meaning
\\	\ character
\' '	character
\" "	character
\? ?	character
\a	Alert or bell
\b	Backspace
\f	Form feed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\ooo	Octal number of one to three digits
\xhhh . . .	Hexadecimal number of one or more digits

Conversión de tipos en C#



La conversión de tipos la realiza automáticamente el compilador o el desarrollador explícitamente

Conversión de tipos implícitos ocurre cuando:

1. Los dos tipos de datos son compatibles.
2. Cuando asignamos un valor de un tipo de datos más pequeño a un tipo de datos más grande.

Convert from Data Type	Convert to Data Type
byte	short, int, long, float, double
short	int, long, float, double
int	long, float, double
long	float, double
float	double

Conversión de tipos en C#

Conversión de tipos Explícita se realiza mediante el operador Cast

Si desea convertir el tipo de datos grande a un tipo de datos pequeño debe hacerlo explícitamente.

En el **Casting de Tipo Explícito** existe la posibilidad de que se pierdan datos o que la conversión no se realice correctamente por algún motivo.

Este es un tipo de conversión inseguro.

Conversión Explícita

code

¿Siempre perdemos datos cuando convertimos un tipo más grande a un tipo más pequeño en C#?

La respuesta es no.

Básicamente depende del valor que estemos convirtiendo y del tamaño del tipo de datos que va a almacenar el valor convertido.

```
int nro = 100;
byte byte_nro = (byte) nro; // Casting
                            // explícito
```

```
int nro2 = 500;
byte byte_nro2 = (byte) nro2; // Casting
                              // explícito
```

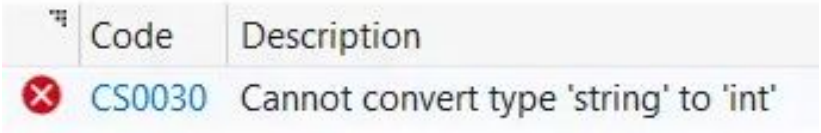
Conversión con métodos auxiliares

El operador de conversión primero verificará la compatibilidad de tipos si son compatibles.

```
string cadena = "100";

int i1 = ( int ) cadena;
```

El compilador descubre que string e int no son compatibles dado que la cadena se usa para almacenar datos textuales que contienen alfanuméricos y int solo contiene datos numéricos.



Convertir métodos auxiliares de clase

NET Framework nos proporciona la clase Convert, con los metodos To…. , el método Parse y el método TryParse.

Method	Description
ToBoolean	It will converts a type to Boolean value
ToChar	It will converts a type to a character value
ToByte	It will converts a value to Byte Value
ToDecimal	It will converts a value to Decimal point value
ToDouble	It will converts a type to double data type
ToInt16	It will converts a type to 16-bit integer
ToInt32	It will converts a type to 32 bit integer
ToInt64	It will converts a type to 64 bit integer
ToString	It will converts a given type to string
ToUInt16	It will converts a type to unsigned 16 bit integer
ToUInt32	It will converts a type to unsigned 32 bit integer
ToUInt64	It will converts a type to unsigned 64 bit integer

InvalidCastException

Esto ocurre cuando no se admite una conversión determinada.

Se produce una excepción para las conversiones siguientes:

- Conversiones de Char **a** Boolean, Single, Double, Decimal o DateTime.
- Conversiones de Boolean, Single, Double, Decimal o DateTime **a** Char.
- Conversiones de DateTime a cualquier otro tipo excepto String.
- Conversiones de cualquier otro tipo, excepto String, **a** DateTime.

Variables



- Contenedores nombrados que almacenan valores.
- Se declaran utilizando la siguiente sintaxis:

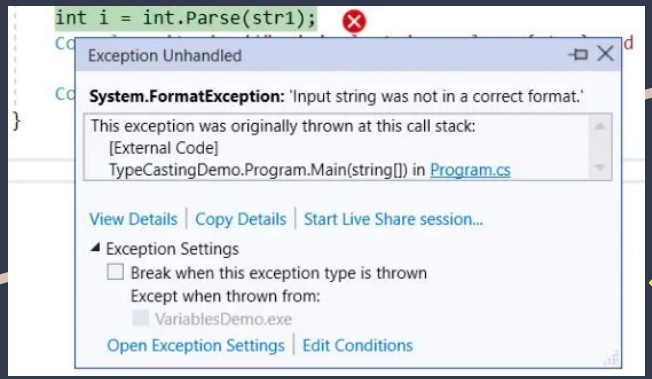
```
tipo_de_dato nombre_de_variable;
```

```
Ejemplo
int numero = 10;

string mensaje = "Hola, mundo!";
```

Pueden ser pasadas por valor o por referencia

Conversión de tipos usando el método Parse()



El método Parse se implementa como un método estático en los tipos de datos de valores integrados.

Conversión de tipos entre tipos no compatibles como int y string

Parse() como los métodos auxiliares de la clase Convert.

```
string str1 = "100";
//Convertir string en int
int i = int.Parse(str1);
```

Si el valor no es compatible con el tipo de destino, se obtendrá un error de tiempo de ejecución

```
string str1 = "Hola";
//Convertir string en int
int i = int.Parse(str1)
```

Usando el método TryParse

Si la conversión no es posible, en tiempo de ejecución obtendremos una excepción,

Si la conversión no es posible, entonces deberíamos mostrar cierta información al usuario y continuar.

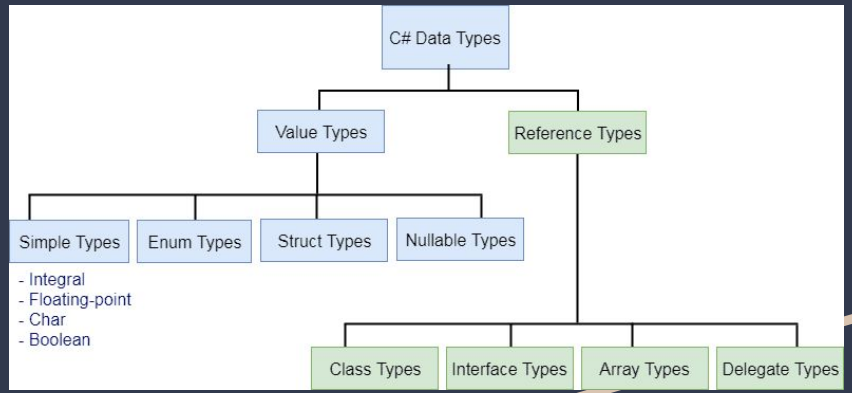
Para hacerlo, la clase de tipo de valor incorporada en C# se proporciona con el método TryParse.

```
bool IsConvertible =
int.TryParse("100", out int sal1);

bool IsConvertible =
int.TryParse("Hola", out int sal2);
```

La primera conversión es exitosa y, por lo tanto, devolverá verdadero y almacenará el valor convertido 100 en la variable sal1. En la segunda la conversión falló, por lo tanto no almacenará nada en la variable sal2 y devolverá falso.

Variables



- El nombre de una variable debe comenzar con una letra o un guión bajo.
- Las variables en C# distinguen entre mayúsculas y minúsculas
- Se pueden construir con dígitos y letras.
- No se permiten símbolos especiales excepto los guiones bajos.
- suma, altura, _valor y abc123, etc. son algunos ejemplos del nombre de la variable

Pueden ser

- **Variable de instancia (no estatica)**
- **Variable estática**
- **La variable constante**
- **Variable de solo lectura**

Tipos de Variables

Variables estáticas y no estáticas en C#

Declaramos la variable explícitamente usando el modificador **static**

El resto son variables no estáticas.

Si declaramos una variable dentro de un bloque estático, entonces esa variable también es una variable estática.

solo podemos tener una copia de una variable estática independientemente de cuántos objetos creemos.

En variables estáticas, **los cambios realizados en un objeto se reflejarán en otros objetos**, ya que las variables estáticas son comunes a todos los objetos de una clase.

A las variables estáticas se puede **acceder directamente usando el nombre de la clase** en C#.

En el ciclo de vida de una clase, **una variable estática se inicializa solo una vez**

Tipos de Variables

Si declaramos una variable usando la palabra clave const, entonces es una variable constante y el valor de la variable constante no se puede modificar una vez después de su declaración.

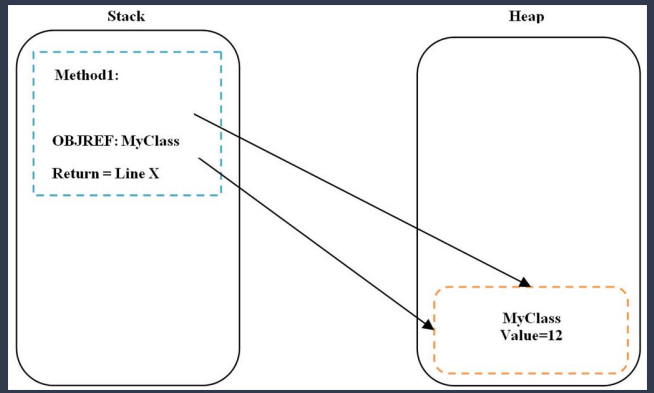
La única diferencia entre una variable estática y una constante es que las variables estáticas se pueden modificar mientras que las variables constantes en C# no se pueden modificar una vez declaradas.

Si declaramos una variable usando la palabra clave **readonly**, se la conoce como variable de solo lectura y estas variables no se pueden modificar como una vez inicializada.

- No es obligatorio inicializar
- Se pueden inicializar bajo el constructor.
- Podemos modificar el valor de la variable de solo lectura solo dentro de un constructor.

Las Variables Locales en C# se declaran dentro del método de una clase, está limitado al método,

Variables por referencia

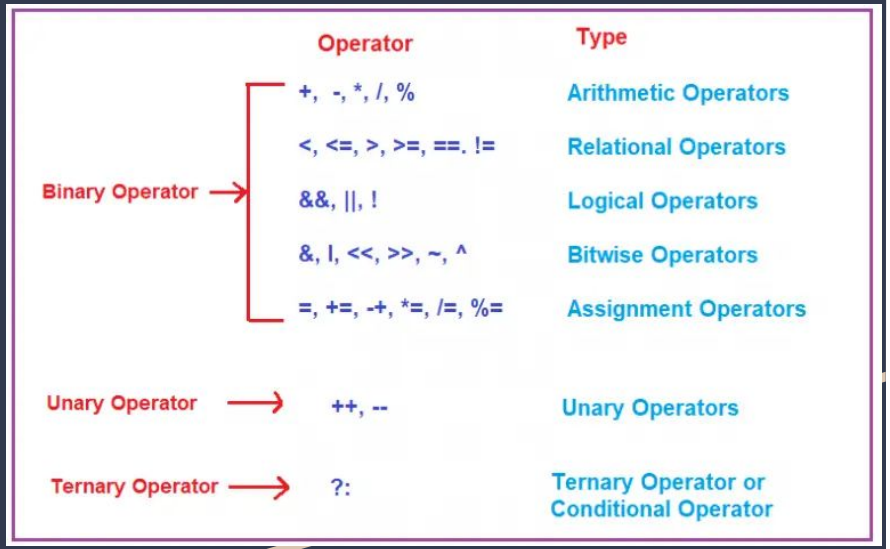


- Representan referencias a ubicaciones de memoria donde se almacenan los datos.
- Algunos ejemplos incluyen:
  - Array: Colecciones de elementos del mismo tipo.
  - Class: Estructuras de datos que encapsulan datos y comportamiento.

```
A v1 = new A();
v1.val = 10;
methodtoshowref(v1);

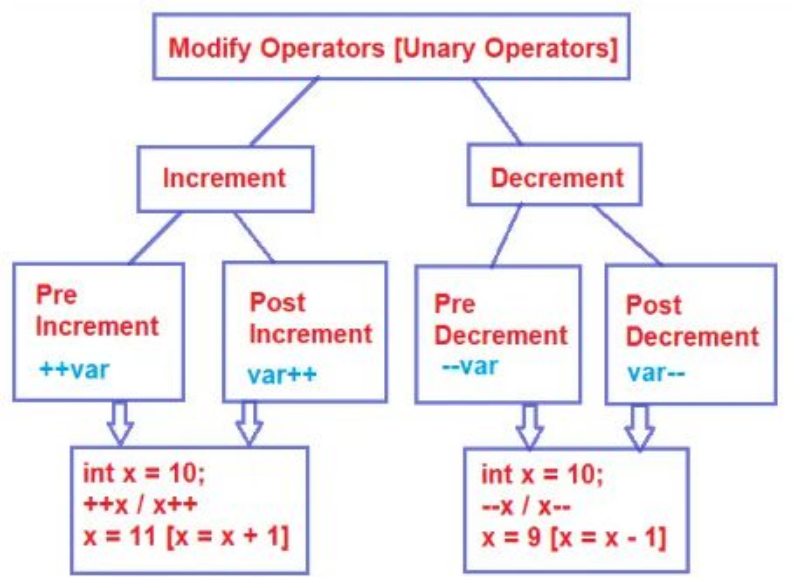
public static void
methodtoshowref(ref A obj)
    {
        obj = new A();;
    }
```

Operadores



- Realizan operaciones sobre valores o variables.
- C# ofrece una variedad de operadores, incluyendo:
  - **Operadores aritméticos:** (+, -, *, /, %)
  - **Operadores relacionales:** (==, !=, <, >, <=, >=)
  - **Operadores lógicos:** (&, |, !)
  - **Operadores de incremento/decremento:** (++,--)
  - **Operadores de asignación:** (=, +=, -=, *=, /=)

Operadores Unarios



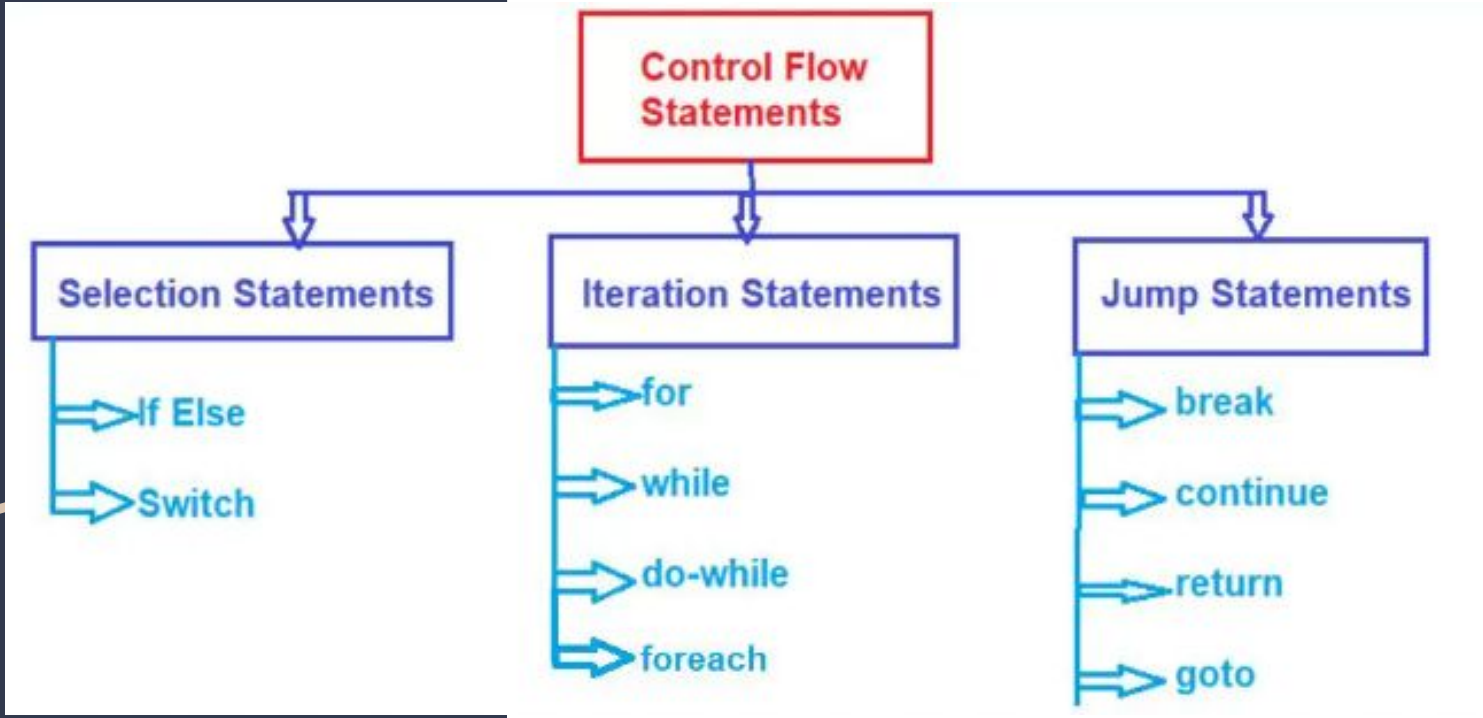
Operadores Ternarios

El operador ternario o también se conocido como operador condicional (? :).(abreviatura de la declaración if-else)

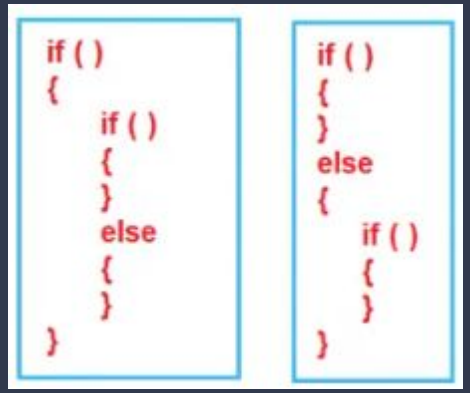
El primer argumento es una comparación,
El segundo es el resultado de una comparación verdadera
El tercero es el resultado de una comparación falsa.

Sintaxis:
¿Condición? primera_expresión: segunda_expresión;

Declaraciones de flujo de control



If-Else




```
if (condition)
    Statement 1;
else if (condition)
    Statement 2;
.
.
.
.
else
    Statement n;
```

Switch

```
switch(variable)
{
    case 1:
            //execute your code
    break;
    case n:
            //execute your code
    break;
    default:
            //execute your code
    break;
}
```

Se permite el anidamiento de declaraciones de case, lo que significa que puede tener declaraciones de case dentro de otro interruptor - reemplaza ||

Microsoft no recomienda las declaraciones de case, motivo más complejo y menos legible.

```
string str = "C#";
switch (str)
{
     case "C#":
     case "Java":
     case "C":
          Console.WriteLine("Esto es un
             lenguaje de
programación");
     break;
     case "MSSQL":
     case "MySQL":
     case "Oracle"
           Console.WriteLine("Esto es
                  Base de datos");
     break;
```

Bucles en C#

Los bucles se dividen principalmente en dos categorías:

Bucles controlados de entrada: Son bucles en los que la condición a probar está presente al comienzo del cuerpo del bucle Ejemplos son el bucle while y el bucle for.

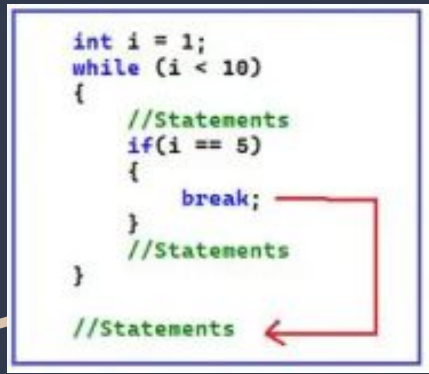
Bucles de salida controlados: Son bucles en los que la condición de prueba está presente al final del cuerpo. Un ejemplo do- while..

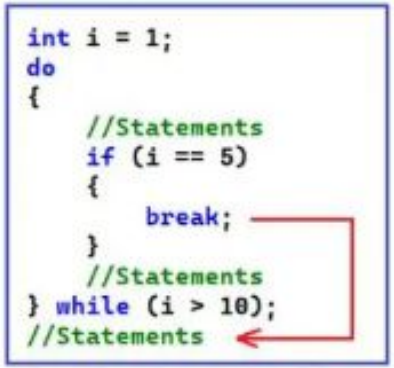
Declaración de Break

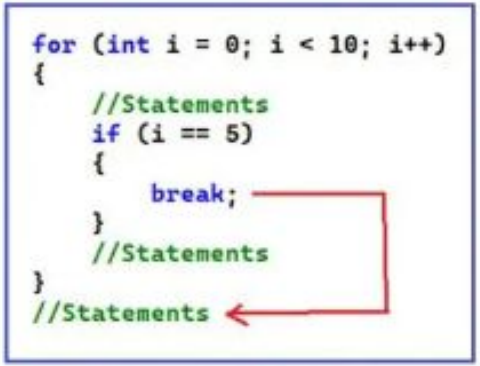
break

return (**Función analizaremos la declaración returns)**

throw (**Manejo de excepciones discutiremos la declaración Throw**)

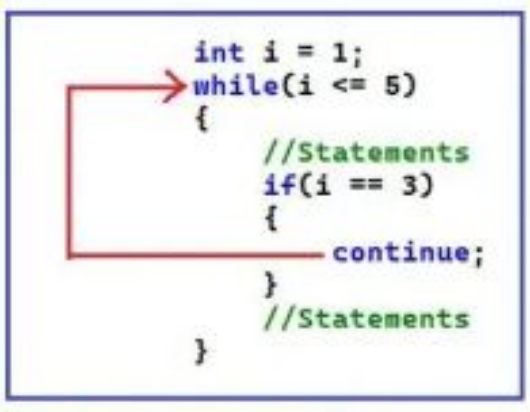


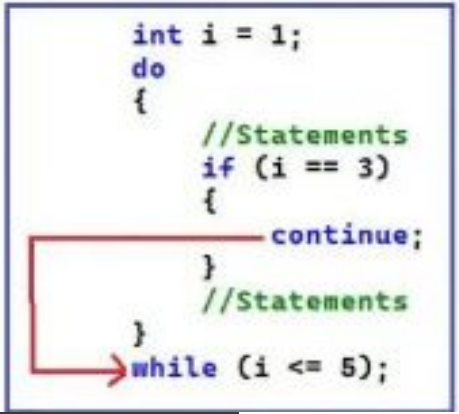


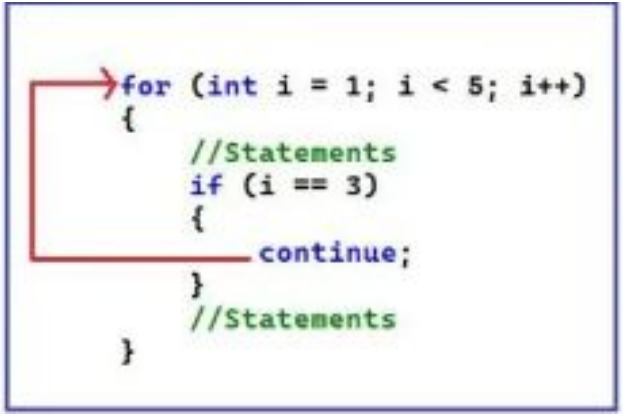


Continue

En C#, continuar es una palabra clave. Al usar la palabra clave continuar, podemos omitir la ejecución de la declaración desde el cuerpo del bucle







Goto

```
goto label;
..
.
label: statement;
```

Goto es una palabra clave y al usarla podemos pasar el control a cualquier parte del programa en el ámbito local. Cuando trabajamos con la declaración goto, se requiere un identificador llamado etiqueta. Cualquier identificador válido seguido de dos puntos se denomina etiqueta. Siempre que trabajamos con una declaración goto, se denomina declaración de flujo de control no estructurado porque rompe la regla del lenguaje de programación estructurada.

```
using System;
namespace Gotodemo
{
    class Program
    {
      static void Main(string[] args)
      {
          Console.WriteLine("Declaración 1: ");
          goto label1;
          Console.WriteLine("Declaración 2 ");
          label1:     //etiqueta:
          Console.WriteLine("Declaración 3 ");
          Console.WriteLine("Finalización del Método: ");
          Console.ReadKey();
      }
    }
}
```

FIN